

Serving DeepSeek Models

Deep Dive into DeepSeek — Chapter 20

Yin Yang

Foundation Books Project

The one rule of this chapter

A checkpoint on disk produces no tokens by itself.

Architecture decides which objects serving must manage.

- A runtime accepts a request, renders its prompt, executes prefill and decode, retains request-local state, and serializes a response.
- For DeepSeek models, MLA gives that state a compressed cache shape and DeepSeekMoE gives it sparse, moved, and placed expert work.
- Low precision and distributed execution turn both of those into payload and partitioning problems, not just modeling choices.

Goal: follow one request from messages to a completed response, using DeepSeek-V2-Lite as a runnable MLA-plus-MoE lane and the pinned DeepSeek-V3 configuration for full-architecture dimensions.

1. From artifact to request state

Roadmap

1. From artifact to request state
2. One request through the model

Roadmap

1. From artifact to request state
2. One request through the model
3. MLA and MoE state

Roadmap

1. From artifact to request state
2. One request through the model
3. MLA and MoE state
4. Cache pressure, scheduling, and precision

Roadmap

1. From artifact to request state
2. One request through the model
3. MLA and MoE state
4. Cache pressure, scheduling, and precision
5. From concept to code

Roadmap

1. From artifact to request state
2. One request through the model
3. MLA and MoE state
4. Cache pressure, scheduling, and precision
5. From concept to code
6. Composing the stack, and what is disclosed

From artifact to request state

Two objects, cleanly separated

Object	What it fixes
Model artifact packet	Weights, configuration, tokenizer, chat template, generation defaults
Serving runtime	Loads the packet, renders requests, runs prefill/decode, owns the cache, batches, samples, responds

- **DeepSeek-V2-Lite** (16B total, 2.4B activated): the runnable MLA-plus-MoE analogue used for executed traces.
- **Pinned DeepSeek-V3** (dim 7168, 61 layers, MLA rank 512, 256 routed experts, 8 activated): the architecture-dimension lane.
- Every section states which lane it is using; no V2-Lite measurement is promoted to a full-V3 result.

One request through the model

Prefill once, decode repeatedly

Phase	Request state	DeepSeek model work
Render	messages → token ids	no transformer work yet

Prefill once, decode repeatedly

Phase	Request state	DeepSeek model work
Render	messages $\rightarrow$ token ids	no transformer work yet
Prefill	T prompt tokens	MLA over prompt positions; dense/MoE FFN

Prefill once, decode repeatedly

Phase	Request state	DeepSeek model work
Render	messages $\rightarrow$ token ids	no transformer work yet
Prefill	T prompt tokens	MLA over prompt positions; dense/MoE FFN
First token	last prefill logits	sample one token

Prefill once, decode repeatedly

Phase	Request state	DeepSeek model work
Render	messages $\rightarrow$ token ids	no transformer work yet
Prefill	T prompt tokens	MLA over prompt positions; dense/MoE FFN
First token	last prefill logits	sample one token
Decode	one new position/step	read+append MLA state; run experts

Prefill once, decode repeatedly

Phase	Request state	DeepSeek model work
Render	messages $\rightarrow$ token ids	no transformer work yet
Prefill	T prompt tokens	MLA over prompt positions; dense/MoE FFN
First token	last prefill logits	sample one token
Decode	one new position/step	read+append MLA state; run experts
Finish	stop or length limit	release state, serialize response

Prefill once, decode repeatedly

Phase	Request state	DeepSeek model work
Render	messages $\rightarrow$ token ids	no transformer work yet
Prefill	T prompt tokens	MLA over prompt positions; dense/MoE FFN
First token	last prefill logits	sample one token
Decode	one new position/step	read+append MLA state; run experts
Finish	stop or length limit	release state, serialize response

$$y_j \rightarrow \text{embed}(y_j) \rightarrow \{\text{MLA}, \text{FFN}/\text{MoE}\}_{\ell=1}^{61} \rightarrow z_{j+1} \rightarrow y_{j+1}$$

Prefill once, decode repeatedly

Phase	Request state	DeepSeek model work
Render	messages $\rightarrow$ token ids	no transformer work yet
Prefill	T prompt tokens	MLA over prompt positions; dense/MoE FFN
First token	last prefill logits	sample one token
Decode	one new position/step	read+append MLA state; run experts
Finish	stop or length limit	release state, serialize response

$$y_j \longrightarrow \text{embed}(y_j) \longrightarrow \{\text{MLA}, \text{FFN}/\text{MoE}\}_{\ell=1}^{61} \longrightarrow z_{j+1} \longrightarrow y_{j+1}$$

This one-token loop is the foundation even when the released MTP module enables speculative decoding as a separate runtime decision.

Registration: configuration becomes an executable model

V3 field	Value	Serving consequence
<code>dim, n_layers</code>	7168, 61	residual width, block count

Registration: configuration becomes an executable model

V3 field	Value	Serving consequence
<code>dim, n_layers</code>	7168, 61	residual width, block count
<code>kv_lora_rank,</code> <code>qk_rope_head_dim</code>	512, 64	MLA cache widths

Registration: configuration becomes an executable model

V3 field	Value	Serving consequence
<code>dim, n_layers</code>	7168, 61	residual width, block count
<code>kv_lora_rank,</code> <code>qk_rope_head_dim</code>	512, 64	MLA cache widths
<code>n_routed_experts,</code> <code>n_activated_experts</code>	256, 8	expert pool and per-token work

Registration: configuration becomes an executable model

V3 field	Value	Serving consequence
<code>dim, n_layers</code>	7168, 61	residual width, block count
<code>kv_lora_rank,</code> <code>qk_rope_head_dim</code>	512, 64	MLA cache widths
<code>n_routed_experts,</code> <code>n_activated_experts</code>	256, 8	expert pool and per-token work
<code>dtype</code>	fp8	weight/GEMM precision path

Registration: configuration becomes an executable model

V3 field	Value	Serving consequence
<code>dim, n_layers</code>	7168, 61	residual width, block count
<code>kv_lora_rank,</code> <code>qk_rope_head_dim</code>	512, 64	MLA cache widths
<code>n_routed_experts,</code> <code>n_activated_experts</code>	256, 8	expert pool and per-token work
<code>dtype</code>	fp8	weight/GEMM precision path

Four gates locate startup failures before a request ever executes: registry → shape → operator → device/dtype. [INSPECTED: DeepSeek-V3 inference/configs/config-671B.json]

MLA and MoE state

MLA: a compressed cache, priced

One absorbed cache entry: $r_{kv} + d_r = 512 + 64 = 576$ scalars per token per layer.

$$M_{\text{token}}^{\text{MLA}} = 61 \cdot 576 \cdot 2 = 70,272 \text{ bytes (BF16)}$$

- Counterfactual full per-head cache: $128(192 + 128) = 40,960$ scalars per token per layer.

MLA: a compressed cache, priced

One absorbed cache entry: $r_{kv} + d_r = 512 + 64 = 576$ scalars per token per layer.

$$M_{\text{token}}^{\text{MLA}} = 61 \cdot 576 \cdot 2 = 70,272 \text{ bytes (BF16)}$$

- Counterfactual full per-head cache: $128(192 + 128) = 40,960$ scalars per token per layer.
- Ratio $40,960/576 \approx 71.1$ — the same public reference model, two representations, one ledger.

MLA: a compressed cache, priced

One absorbed cache entry: $r_{kv} + d_r = 512 + 64 = 576$ scalars per token per layer.

$$M_{\text{token}}^{\text{MLA}} = 61 \cdot 576 \cdot 2 = 70,272 \text{ bytes (BF16)}$$

- Counterfactual full per-head cache: $128(192 + 128) = 40,960$ scalars per token per layer.
- Ratio $40,960/576 \approx 71.1$ — the same public reference model, two representations, one ledger.

This accounting comparison lives entirely inside the released V3 implementation; it is not a claim about every deployment's cache choice. [DERIVED: DeepSeek-V3 configuration, arithmetic ledger]

DeepSeekMoE: sparse compute, moved and placed

Component	Serving role
DeepGEMM	Grouped expert-matrix compute

DeepSeekMoE: sparse compute, moved and placed

Component	Serving role
DeepGEMM	Grouped expert-matrix compute
DeepEP	Dispatch rows to expert owners, combine results back

DeepSeekMoE: sparse compute, moved and placed

Component	Serving role
DeepGEMM	Grouped expert-matrix compute
DeepEP	Dispatch rows to expert owners, combine results back
EPLB	Map logical experts to physical replicas

DeepSeekMoE: sparse compute, moved and placed

Component	Serving role
DeepGEMM	Grouped expert-matrix compute
DeepEP	Dispatch rows to expert owners, combine results back
EPLB	Map logical experts to physical replicas

For B token rows and top-8 routing, the router creates

$$N_{\text{routed rows}} = 8B$$

routed rows, plus shared-expert work for the original B rows.

DeepSeekMoE: sparse compute, moved and placed

Component	Serving role
DeepGEMM	Grouped expert-matrix compute
DeepEP	Dispatch rows to expert owners, combine results back
EPLB	Map logical experts to physical replicas

For B token rows and top-8 routing, the router creates

$$N_{\text{routed rows}} = 8B$$

routed rows, plus shared-expert work for the original B rows.

No single component owns routing, movement, compute, and placement at once — that separation is the serving picture of sparse activation.

Cache pressure, scheduling, and precision

Paged cache and scheduling: the same budget, two directions

$$N_{\text{page}}(T) = \left\lceil \frac{T}{P} \right\rceil$$

Req.	Positions	Pages/Slack
A	2000	32 / 48
B	4100	65 / 60

slots of slack at 70,272 bytes each.

97 pages, 108

Instinct	Tradeoff
Decode priority	large MLA-prefill may wait
Chunked prefill	more scheduling boundaries
Prefill	ready decode may wait
throughput	

No

scheduler name fixes admission priority or fairness by itself.

Precision belongs to the payload, not the model name

Surface	What can vary independently
Weights	FP8 with per-block scale layout
Activations/GEMM	quantized operands, higher-precision accumulation
Expert communication	DeepEP can move low-precision payload plus scales
MLA cache	BF16 rows, or a backend-specific mixed FP8-plus-scale layout

“The model uses FP8” is not a complete serving description — every quantized block must keep the scale granularity that created it.

From concept to code

From concept to code: two caches, not one

The reference implementation registers exactly the two objects MLA's absorbed path predicts — and the naive path registers full per-head views instead.

```
from DeepSeek-V3/inference/model.py
```

```
if attn_impl == "naive":
    self.register_buffer("k_cache", torch.zeros(..., self.qk_head_dim), ...)
    self.register_buffer("v_cache", torch.zeros(..., self.v_head_dim), ...)
else:
    self.register_buffer("kv_cache", torch.zeros(..., self.kv_lora_rank), ...)
    self.register_buffer("pe_cache", torch.zeros(..., self.qk_rope_head_dim), ...)
```

From concept to code: two caches, not one

The reference implementation registers exactly the two objects MLA's absorbed path predicts — and the naive path registers full per-head views instead.

```
from DeepSeek-V3/inference/model.py
```

```
if attn_impl == "naive":
    self.register_buffer("k_cache", torch.zeros(..., self.qk_head_dim), ...)
    self.register_buffer("v_cache", torch.zeros(..., self.v_head_dim), ...)
else:
    self.register_buffer("kv_cache", torch.zeros(..., self.kv_lora_rank), ...)
    self.register_buffer("pe_cache", torch.zeros(..., self.qk_rope_head_dim), ...)
```

`kv_cache` is the latent row at width `kv_lora_rank`; `pe_cache` is the RoPE-sensitive row at width `qk_rope_head_dim`. Serving frameworks that support MLA allocate, page, and read exactly this pair. [INSPECTED: DeepSeek-V3 inference/model.py]

Composing the stack, and what is disclosed

One open stack, layer by layer

Layer	Public realization
Artifact	DeepSeek-V2-Lite-Chat or DeepSeek-V3 model packet
Serving runtime	SGLang, vLLM, Transformers, or another open integration
Attention	runtime-native MLA path or FlashMLA integration
Expert compute/- comm/placement	DeepGEMM, DeepEP, EPLB

A direct runtime, a scaled serving engine, and a distributed V3 target are three different deployment envelopes — none should be read as evidence for the other two.

The disclosed V3/R1 online system

Phase	Deployment unit	Routed experts	Per-GPU assignment
Prefill	4 nodes	EP32, 32 redundant	9 routed + 1 shared

The disclosed V3/R1 online system

Phase	Deployment unit	Routed experts	Per-GPU assignment
Prefill	4 nodes	EP32, 32 redundant	9 routed + 1 shared
Decode	18 nodes	EP144, 32 redundant	2 routed + 1 shared

The disclosed V3/R1 online system

Phase	Deployment unit	Routed experts	Per-GPU assignment
Prefill	4 nodes	EP32, 32 redundant	9 routed + 1 shared
Decode	18 nodes	EP144, 32 redundant	2 routed + 1 shared

Dual-batch overlap hides dispatch/combine communication behind independent computation; three separate balancers address prefill, decode, and expert-parallel skew.

The disclosed V3/R1 online system

Phase	Deployment unit	Routed experts	Per-GPU assignment
Prefill	4 nodes	EP32, 32 redundant	9 routed + 1 shared
Decode	18 nodes	EP144, 32 redundant	2 routed + 1 shared

Dual-batch overlap hides dispatch/combine communication behind independent computation; three separate balancers address prefill, decode, and expert-parallel skew.

Dated 24-hour window: 608B input tokens (56.3% cache hits), 168B output tokens, 20–22 tokens/s average output speed. [REPORTED: DeepSeek Open Source Week Day 6 overview]

Concept-to-code map for the chapter

- **model construction from configuration** → `DeepSeek-V3/inference/configs/config_671B.json`
- **MLA's two cache buffers** → `DeepSeek-V3/inference/model.py`
- **decode-time cache read** → `FlashMLA/flash_mla/flash_mla_interface.py`
- **grouped expert dispatch/combine** → `DeepEP/csrc/python_api.cpp`
- **logical scheduler reference** →
`code/chapter_serving_deepseek_models/serving_schedule_reference.py`

Render, prefill, decode — with a compressed cache and sparse, moved, placed expert work.

Next: FlashMLA opens the attention kernel schedule, DeepGEMM opens the matrix kernels, and DeepEP/EPLB open the communication and placement algorithms this chapter only assigned ownership to.